DMA

1. DMA Overview

DMA (Direct Memory Access) is a data transfer technology that works without CPU intervention.

Think of it this way: The CPU is like a head chef. Without DMA, even the menial tasks of buying vegetables, washing vegetables, and cutting vegetables all have to be done by the chef themselves, too busy to cook. With DMA, it's like hiring an assistant—the chef only needs to tell the assistant "wash and cut those vegetables and bring them over", then can immediately focus on cooking. **DMA is that assistant**, a separate hardware channel dedicated to transporting data between memory and peripherals. The CPU only needs to say "move from A to B", then can go handle other tasks.

In this experiment, you write a UART serial transmission program. Without DMA, CPU has to personally write each byte to UART's TX register, write one, wait for hardware to finish sending, then write the next—sending 100 bytes ties up CPU 100 times. With DMA, CPU dumps all data to be sent into a "ring buffer" in memory, says "it's all yours", immediately returns to handle other tasks, DMA quietly moves data one by one to UART in the background—sending 100 bytes, CPU only participates 1 time.

This is DMA's core value: **Liberating CPU from repetitive, mechanical data movement, allowing CPU to do more meaningful things.**

This experiment demonstrates UART combined with DMA loopback test: use a Dupont wire to short GPIO14 (TX) and GPIO4 (RX). Data goes from TX via DMA, through Dupont wire back to RX, then DMA automatically receives—complete "send→loopback receive" visible in the same serial monitor, no additional hardware needed.

Key Differences with/without DMA:

	Without DMA (polling/interrupt mode)	With DMA (this experiment)
CPU Involvement	CPU writes to TX FIFO byte by byte	CPU writes to ring buffer once
Throughput	Limited by CPU response speed	Approaches hardware maximum bandwidth
Blocking	CPU long wait for large data	CPU returns immediately, non-blocking
Use Case	Small debug logs, AT commands	High-frequency sensor data streams, Wi-Fi log forwarding

Typical Application Scenarios:

Application Type	Example
Audio streaming	I2S microphone recording, audio playback
Image transmission	Camera OV2640 data movement, LCD screen refresh
High-speed communication	UART large frame transmit/receive, SPI Flash batch read/write
Data acquisition	High-speed ADC waveform acquisition, DAC arbitrary waveform playback
Storage movement	SD card read/write, PSRAM↔Flash data transfer

---

2. Core Concepts

2.1 UART Transmit Data Path

```
Without DMA transmit:
  CPU ──byte by byte──→ UART TX FIFO (128 bytes) ──→ TX shift register ──→ GPIO pin

With DMA transmit:
  CPU ──all at once──→ TX ring buffer (RAM, 1024 bytes)
                          ↓ DMA auto-moves
                      UART TX FIFO (128 bytes)
                          ↓
                      TX shift register ──→ GPIO pin
```

DMA's liberating effect: CPU only needs to call `uart_write_bytes()` once to write data to RAM ring buffer. Subsequent "buffer→FIFO→shift register→pin" all automatically completed by DMA hardware.

2.2 Ring Buffer

```
Ring buffer diagram (1024 bytes):

   ┌───┬───┬───┬───┬───┬───┬───┬───┬───┬───┐
   | D | a | t | a |   |   |   |   |   |   | ...
   └───┴───┴───┴───┴───┴───┴───┴───┴───┴───┘
     ↑                       ↑
    Read pointer (DMA takes)  Write pointer (CPU fills)

When write pointer catches read pointer → buffer full → block waiting for DMA consumption
When read pointer catches write pointer → buffer empty → DMA idle waiting for CPU fill
```

`uart_driver_install()` sets `BUF_SIZE = 1024`. At 115200bps rate, can send approximately 11,520 bytes per second. 1024-byte buffer is enough to buffer about 89ms of data, leaving ample scheduling margin.

2.3 UART Configuration Parameters

```c
uart_config_t uart_config = {
    // Baud rate: 115200 bps
    .baud_rate  = 115200,
    .data_bits  = UART_DATA_8_BITS,    // 8 data bits
    .parity     = UART_PARITY_DISABLE, // No parity
    .stop_bits  = UART_STOP_BITS_1,    // 1 stop bit
    // Hardware flow control: off
    .flow_ctrl  = UART_HW_FLOWCTRL_DISABLE,
    .source_clk = UART_SCLK_DEFAULT,   // Default clock source
};
```

Standard configuration **115200-8-N-1**, no hardware flow control (RTS/CTS).

2.4 Actual Pins Used in This Experiment

> **Note**: This experiment uses **GPIO14 (TX)** and **GPIO4 (RX)** in code, not GPIO17/GPIO18 as recorded in architecture documentation. This is because GPIO17/18 are occupied by other peripherals on some core boards. The following analysis uses GPIO14/GPIO4 as per actual code.

---

3. Project Architecture and Flow

```
app_main()
  ├─ uart_dma_init()              Initialize UART1 + DMA channel
  |    ├─ uart_param_config()        Configure: 115200/8N1, no flow control
  |    ├─ uart_set_pin()             Bind: TX=GPIO14, RX=GPIO4
  |    └─ uart_driver_install()      Install: TX/RX each 1KB ring buffer, flags=0 (DMA)
  |
  └─ while(1) Main Loop
       ├─ uart_dma_loopback(test_data)   DMA send + loopback receive
       |    ├─ uart_write_bytes()           Write to TX ring buffer → return immediately
       |    |     ↓ (DMA background moves to GPIO14 pin)
```

```
      |   |     TX → Dupont wire → RX
      |   |     ↓ (DMA auto-moves RX pin data to RX ring buffer)
      |   ├─ vTaskDelay(wait for transfer complete)
      |   └─ uart_read_bytes()           Read from RX ring buffer → print verification
      |
      └─ vTaskDelay(1000ms)            Once per second
```

DMA Loopback Timing:

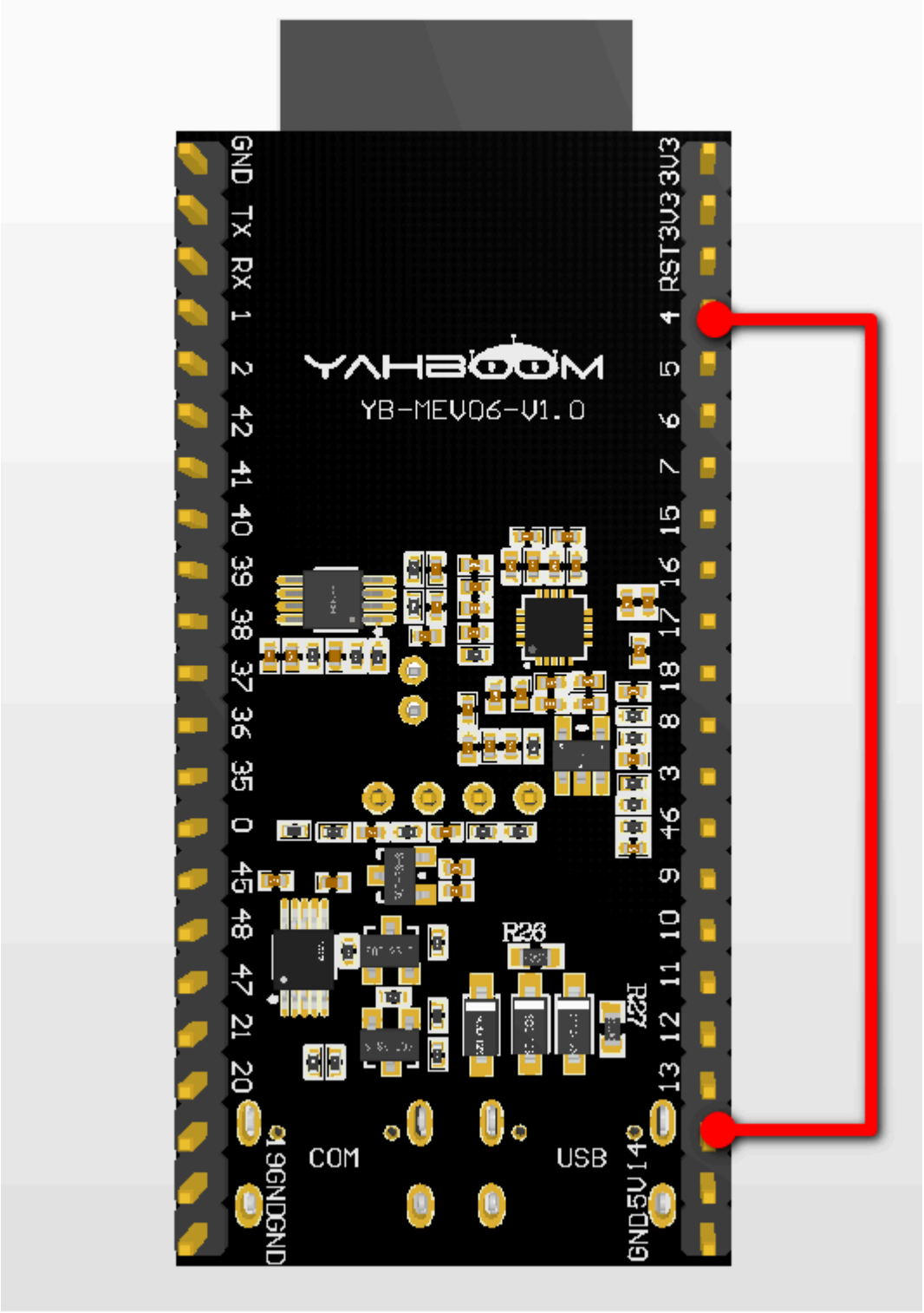
```
Timeline:
  0ms       Call uart_write_bytes() → data enters TX ring buffer → CPU returns immediately
  0~1ms     DMA moves: TX buffer → UART TX FIFO → GPIO14 output
             ┌─ Dupont wire ─┐
            GPIO14(TX) ──→ GPIO4(RX)
  1~2ms     DMA moves: GPIO4 → UART RX FIFO → RX ring buffer
  ~2ms      uart_read_bytes() extracts RX buffer data → print "Loopback received"

CPU is completely idle during 0~2ms transfer (relative to UART send/receive), can process
other tasks
```

---

4. Hardware Dependencies

Hardware Connection:

Only need one Dupont wire (female to female), short GPIO14 and GPIO4 to form loopback:




```
GPIO14 (TX) ——[Dupont wire]—— GPIO4 (RX)
```

> Principle: Data goes out from TX pin, through Dupont wire directly into RX pin, DMA automatically receives into RX ring buffer. This is called "loopback test", no USB-TTL module needed, one wire handles it. Note Dupont wire must be on the same UART—this experiment uses UART1 (GPIO14/GPIO4).

Dependencies: `freertos`, `driver/uart`, `driver/gpio`, `esp_log`

5. Source Code Analysis

5.1 Macro Definitions and Test Data

First give UART1, pin numbers, buffer size constants readable names (avoiding magic numbers like "1", "14", "4", "1024" scattered throughout code), then prepare a test string to send.

```
#define TAG      "UART_DMA"                          // Log tag
// Use UART1 (ESP32-S3 has 3: UART0/1/2)
#define UART_NUM  UART_NUM_1
#define TXD_PIN   GPIO_NUM_14
#define RXD_PIN   GPIO_NUM_4
// Ring buffer size (bytes)
#define BUF_SIZE  1024

static const char *test_data =
    "Hello ESP32-S3 DMA UART! This is a DMA transmission test.\r\n";
```

> **Why use UART1 instead of UART0?** UART0 is used for ESP-IDF log output by default (`ESP_LOGI` / `printf`). Using UART1 avoids conflict with system logs while not affecting IDE serial monitor.

5.2 uart_dma_init() — DMA Driver Initialization

This function does three things—configure serial port parameters (baud rate, data bits, etc.), specify which two GPIOs for TX and RX, install driver and enable DMA channel. After three steps, UART1 has DMA auto-send capability.

```
static void uart_dma_init(void)
{
    /* Configure serial port: 115200-8-N-1, no hardware flow control */
    uart_config_t uart_config = {
        .baud_rate  = 115200,
        .data_bits  = UART_DATA_8_BITS,
        .parity     = UART_PARITY_DISABLE,
        .stop_bits  = UART_STOP_BITS_1,
        .flow_ctrl  = UART_HW_FLOWCTRL_DISABLE,
        .source_clk = UART_SCLK_DEFAULT,
    };
    ESP_ERROR_CHECK(uart_param_config(UART_NUM, &uart_config));

    /* Bind TX/RX pins */
    ESP_ERROR_CHECK(uart_set_pin(UART_NUM, TXD_PIN, RXD_PIN,
                                 UART_PIN_NO_CHANGE, UART_PIN_NO_CHANGE));

    /* Install driver: 1KB ring buffers TX/RX, queue depth 10, flags=0 (DMA on) */
    ESP_ERROR_CHECK(uart_driver_install(UART_NUM, BUF_SIZE, BUF_SIZE, 10, NULL, 0));

    ESP_LOGI(TAG, "UART DMA initialized, TX=GPIO%d, RX=GPIO%d", TXD_PIN, RXD_PIN);
}
```

> **Meaning of last parameter `flags=0` in `uart_driver_install()`**: Passing 0 means use default configuration (DMA enabled). Passing `UART_DRIVER_NO_DMA` disables DMA and uses interrupt mode instead.
>
> **Queue depth 10**: UART driver internally uses FreeRTOS queues to manage events (TX complete, RX arrived, etc.). 10 is a reasonable default, reducing RAM usage without losing events.

5.3 uart_dma_loopback() — DMA Send + Loopback Receive

Core of the entire loopback test is here, three steps: call `uart_write_bytes()` to fill data into TX ring buffer and hand to DMA; wait a bit for data to go from GPIO14 out through Dupont wire back to GPIO4, then DMA receives into RX ring buffer; finally `uart_read_bytes()` extracts data from RX ring buffer and prints for verification.

```
static void uart_dma_loopback(const char *data, int len)
{
    /* Write to TX ring buffer, DMA sends in background */
    int bytes_sent = uart_write_bytes(UART_NUM, data, len);
    ESP_LOGI(TAG, "DMA sent %d bytes", bytes_sent);

    /* Wait for data to go TX → jumper wire → RX (~87μs per byte at 115200bps) */
    int wait_ms = (len * 10 * 1000) / 115200 + 5;
    vTaskDelay(pdMS_TO_TICKS(wait_ms));

    /* Read loopback data from RX ring buffer */
    uint8_t rx_buf[256] = {0};
    int rx_len = uart_read_bytes(UART_NUM, rx_buf, sizeof(rx_buf) - 1, pdMS_TO_TICKS(100));

    if (rx_len > 0) {

        /* Loopback OK: print received data */
        rx_buf[rx_len] = '\0';
        ESP_LOGI(TAG, "Loopback received %d bytes: %s", rx_len, (char *)rx_buf);
    } else if (rx_len == 0) {

        /* Timeout: jumper wire may be loose */
        ESP_LOGW(TAG, "Loopback timeout — no data received, check GPIO14-GPIO4 wire!");
    } else {

        /* Read error */
        ESP_LOGE(TAG, "Loopback read error: %d", rx_len);
    }
}
```

DMA mode behavior of `uart_write_bytes()`:

1. Check if TX ring buffer has enough free space (`BUF_SIZE - used space`)
2. If yes: copy `data` to ring buffer → return copied byte count immediately
3. If no (buffer full): block waiting for DMA to consume and free space → then copy
4. DMA in background: ring buffer → UART TX FIFO → TX shift register → GPIO14 pin

Calculation of `wait_ms`: At 115200bps, each bit takes ~8.7μs, one byte = 1 start bit + 8 data bits + 1 stop bit = 10 bits, so each byte takes ~87μs. `len * 10 * 1000 / 115200` calculates theoretical transfer time, extra 5ms added as safety margin.

DMA mode behavior of `uart_read_bytes()`: When RX pin has data arriving, DMA automatically moves data from UART RX FIFO to RX ring buffer. `uart_read_bytes()` reads from this buffer; if buffer is empty, waits `timeout` milliseconds. If Dupont wire isn't connected properly, RX buffer stays empty, `uart_read_bytes()` times out and returns 0, triggering warning.

> **Return value** `bytes_sent` **is actually bytes the buffer has received**, not necessarily bytes DMA has finished sending. Since DMA is non-blocking, `bytes_sent` equal to `len` means data was successfully submitted to send queue.

5.4 app_main() — Main Entry

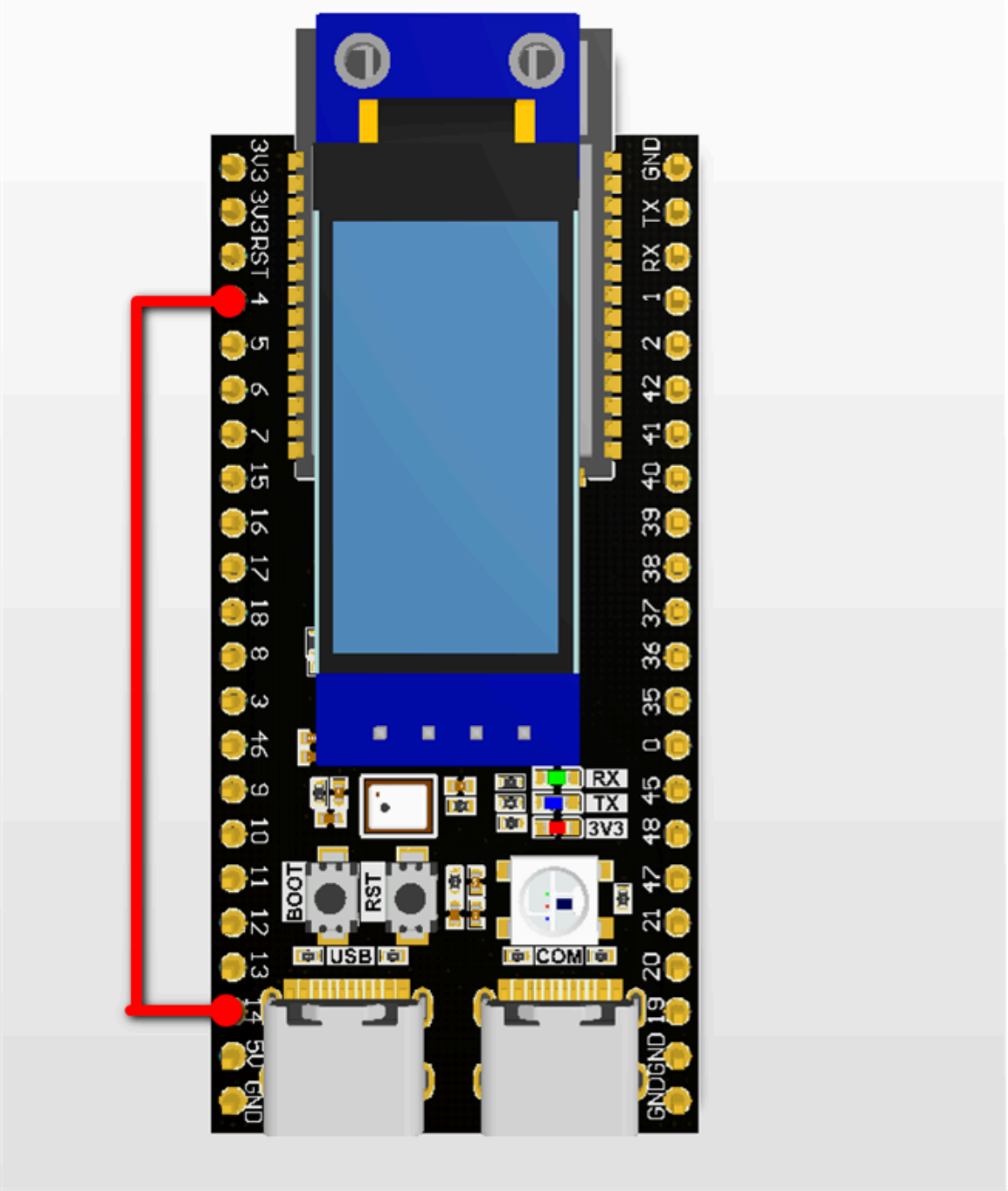
Initialize DMA serial, wait 100ms for peripheral to stabilize, then enter infinite loop, call `uart_dma_loopback()` every 1 second to send and loopback receive.

```
void app_main(void)
{
    /* UART DMA loopback test experiment */
    ESP_LOGI(TAG, "========================================");
    ESP_LOGI(TAG, "ESP32-S3 UART DMA Loopback Experiment");
    ESP_LOGI(TAG, "========================================");

    /* Init UART DMA */
    uart_dma_init();
    vTaskDelay(pdMS_TO_TICKS(100));   // Wait for stabilization

    int count = 0;
    while (1) {
        count++;
        /* DMA loopback #X */
        ESP_LOGI(TAG, "--- DMA loopback #%d ---", count);
        uart_dma_loopback(test_data, strlen(test_data));
        vTaskDelay(pdMS_TO_TICKS(1000));   // Once per second
    }
}
```

6. Operation Flow

6.1 Hardware Preparation

1. One Dupont wire (female to female)
2. Short GPIO14 and GPIO4 with Dupont wire:




```
GPIO14 (TX) ——[Dupont wire]—— GPIO4 (RX)
```

3. Connect ESP32-S3 to computer with USB cable, open IDE serial monitor. No additional USB-TTL module needed.

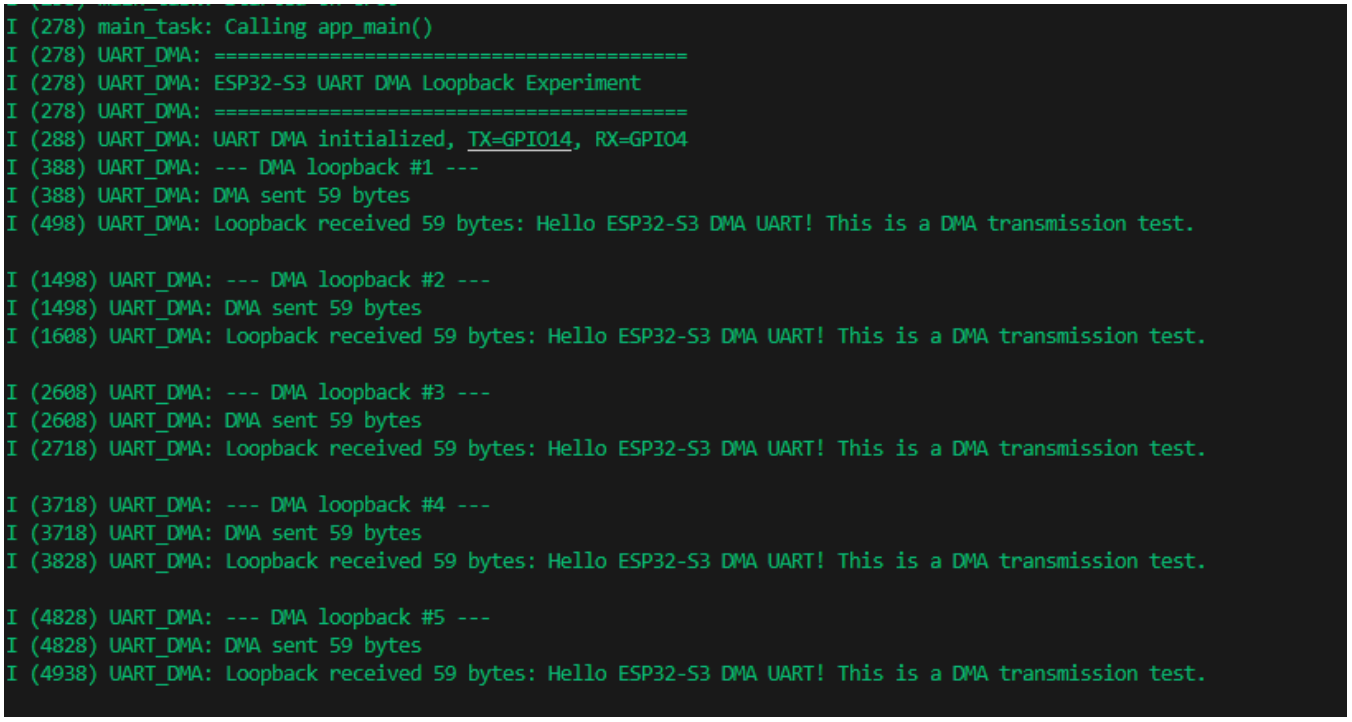
6.2 Build and Flash

> **For detailed flashing process, see: 1. Before Development → 3. Build and Flash**

6.3 Normal Output

All output in IDE serial monitor (UART0), no additional serial assistant needed:

```
I (xxx) UART_DMA: ========================================
I (xxx) UART_DMA: ESP32-S3 UART DMA Loopback Experiment
I (xxx) UART_DMA: ========================================
I (xxx) UART_DMA: UART DMA initialized, TX=GPIO14, RX=GPIO4
I (xxx) UART_DMA: --- DMA loopback #1 ---
I (xxx) UART_DMA: DMA sent 54 bytes
I (xxx) UART_DMA: Loopback received 54 bytes: Hello ESP32-S3 DMA UART! This is a DMA
transmission test.
I (xxx) UART_DMA: --- DMA loopback #2 ---
I (xxx) UART_DMA: DMA sent 54 bytes
I (xxx) UART_DMA: Loopback received 54 bytes: Hello ESP32-S3 DMA UART! This is a DMA
transmission test.
```



> **Verification logic**: If "Loopback received" print content exactly matches sent content, both DMA TX channel (send) and RX channel (receive) are working normally, Dupont wire loopback is unobstructed.

---

7. Common Issues

Phenomenon	Cause	Solution
"Loopback timeout" warning	Dupont wire not connected properly or loose contact	Reinsert Dupont wire firmly, confirm GPIO14 and GPIO4 are shorted reliably
Loopback data garbled	Baud rate mismatch or Dupont wire interference	Confirm baud rate is 115200 in code, try shorter Dupont wire

Phenomenon	Cause	Solution
DMA send returns 0	TX ring buffer full (DMA abnormal stop)	Increase BUF_SIZE or reduce send frequency
`uart_driver_install` fails	UART1 already occupied by other code	Check if any other component uses UART_NUM_1
Logs and DMA data mixed	UART0 and UART1 share pins	Confirm UART0 TX=GPIO43, UART1 TX=GPIO14
TX FIFO overflow	Send speed > baud rate capacity	Reduce send frequency / increase baud rate / increase buffer